Definition 3. A (stationary) policy π is ε-suboptimal if $J(\pi) \geq J(\pi^\star) - \varepsilon$.

Alternatively, if the learning algorithm always uses a policy that is "nearly" deterministic (but with some probability of exploration), then we might not care about hackability resulting from very stochastic policies, leading us to consider the following definition:

Definition 4. A (stationary) policy π is δ-deterministic if $\forall s \in S\ \exists a \in A : \mathbb{P}(\pi(s) = a) \geq \delta$.

Unfortunately, both of these sets contain open subsets, which means they are subject to Theorem 1.

Corollary 2. *In any $MDP \setminus \mathcal{R}$, any pair of reward functions that are unhackable and non-trivial on the set of all ε-suboptimal policies ($\varepsilon > 0$) Π^ε are equivalent on Π^ε, and any pair of reward functions that are unhackable and non-trivial on the set of all δ-deterministic policies ($\delta < 1$) Π^δ are equivalent on Π^δ.*

Intuitively, Theorem 1 can be applied to any policy set with "volume" in policy space.

5.2 Finite Policy Sets

Having established that interesting unhackability is impossible relative to the set of all policies, we now turn our attention to the case of *finite* policy sets. Note that this includes the set of all deterministic policies, since we restrict our analysis to finite MDPs. Surprisingly, here we find that non-trivial non-equivalent unhackable reward pairs *always* exist.

Theorem 2. *For any $MDP \setminus \mathcal{R}$, any finite set of policies $\hat{\Pi}$ containing at least two π, π' such that $\mathcal{F}(\pi) \neq \mathcal{F}(\pi')$, and any reward function $\mathcal{R}_1$, there is a non-trivial reward function $\mathcal{R}_2$ such that $\mathcal{R}_1$ and $\mathcal{R}_2$ are unhackable but not equivalent.*

This proof proceeds by finding a path from $\mathcal{R}_1$ to another reward function $\mathcal{R}_3$ that is hackable with respect to $\mathcal{R}_1$. Along the way to reversing one of $\mathcal{R}_1$'s inequalities, we must encounter a reward function $\mathcal{R}_2$ that instead replaces it with equality. In the case that $\dim(\hat{\Pi}) = 3$, we can visualize moving along this path as rotating the contour lines of a reward function defined on the space containing the policies' discounted state-action occupancies, see Figure 4. This path can be constructed so as to avoid any reward functions that produce trivial policy orderings, thus guaranteeing $\mathcal{R}_2$ is non-trivial. For a *simplification* to exist, we require some further conditions, as established by the following theorem:

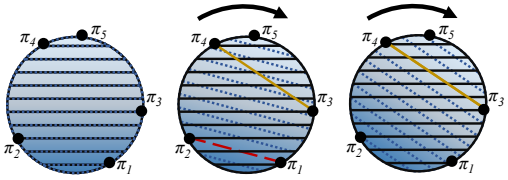


Figure 4: An illustration of the state-action occupancy space with a reward function defined over it. Points correspond to policies' state-action occupancies. Shading intensity indicates expected reward. Rotating the reward function to make $J(\pi_3) > J(\pi_4)$ passes through a reward function that sets $J(\pi_1) = J(\pi_2)$. Solid black lines are contour lines of the original reward function, dotted blue lines are contour lines of the rotated reward function.

Theorem 3. *Let $\hat{\Pi}$ be a finite set of policies, and $\mathcal{R}_1$ a reward function. The following procedure determines if there exists a non-trivial simplification of $\mathcal{R}_1$ in a given $MDP \setminus \mathcal{R}$:*

1. *Let $E_1 \dots E_m$ be the partition of $\hat{\Pi}$ where π, π' belong to the same set iff $J(\pi) = J(\pi')$.*
2. *For each such set E_i, select a policy $\pi_i \in E_i$ and let Z_i be the set of vectors that is obtained by subtracting $\mathcal{F}(\pi_i)$ from each element of $\mathcal{F}(E_i)$.*

Then there is a non-trivial simplification of $\mathcal{R}$ iff $\dim(Z_1 \cup \dots \cup Z_m) \leq \dim(\mathcal{F}(\hat{\Pi})) - 2$, where $\dim(S)$ is the number of linearly independent vectors in S.

The proof proceeds similarly to Theorem 2. However, in Theorem 2 it was sufficient to show that there are no trivial reward functions along the path from $\mathcal{R}_1$ to $\mathcal{R}_3$, whereas here we additionally need that if $J(\pi) = J(\pi')$ then $J'(\pi) = J'(\pi')$ for all functions $\mathcal{R}_2$ on the path — this is what the extra conditions ensure.

Theorem 3 is opaque, but intuitively, the cases where $\mathcal{R}_1$ cannot be simplified are those where $\mathcal{R}_1$ imposes many different equality constraints that are difficult to satisfy simultaneously. We can think

of $\dim(\mathcal{F}(\Pi))$ as measuring how diverse the behaviours of policies in policy set Π are. Having a less diverse policy set means that a given policy ordering imposes fewer constraints on the reward function, creating more potential for simplification. The technical conditions of this proof determine when the diversity of Π is or is not sufficient to prohibit simplification, as measured by $\dim(Z_1 \cup \cdots \cup Z_m)$.

Projecting E_i to Z_i simply moves these spaces to the origin, so that we can compare the directions in which they vary (i.e. their span). By assumption, $E_i \cap E_j = \{\}$, but $\mathrm{span}(Z_i) \cap \mathrm{span}(Z_j)$ will include the origin, and may also contain linear subspaces of dimension greater than 0. This is the case exactly when there are a pair of policies in E_i and a pair of policies in E_j that differ by the same visit counts, for example, when the environment contains an obstacle that could be circumnavigated in several different ways (with an impact on visit counts, but no impact on reward), and the policies in E_i and E_j both need to circumnavigate it before doing something else. Roughly speaking, $\dim(Z_1 \cup \cdots \cup Z_m)$ is large when either (i) we have very large and diverse sets of policies in $\hat{\Pi}$ that get the same reward according to $\mathcal{R}$, or (ii) we have a large number of different sets of policies that get the same reward according to $\mathcal{R}$, and where there are different kinds of diversity in the behaviour of the policies in each set. There are also intuitive special cases of Theorem 3. For example, as noted before, if E_i is a singleton then Z_i has no impact on $\dim(Z_1 \cup \cdots \cup Z_m)$. This implies the following corollary:

Corollary 3. *For any finite set of policies $\hat{\Pi}$, any environment, and any reward function $\mathcal{R}$, if $|\hat{\Pi}| \geq 2$ and $J(\pi) \neq J(\pi')$ for all $\pi, \pi' \in \hat{\Pi}$ then there is a non-trivial simplification of $\mathcal{R}$.*

A natural question is whether any reward function is guaranteed to have a non-trivial simplification on the set of all deterministic policies. As it turns out, this is not the case. For concreteness, and to build intuition for this result, we examine the set of deterministic policies in a simple $MDP \setminus \mathcal{R}$ with $S = \{0, 1\}, A = \{0, 1\}, T(s, a) = a, I = \{0 : 0.5, 1 : 0.5\}, \gamma = 0.5$. Denote π_{ij} the policy that takes action i from state 0 and action j from state 1. There are exactly four deterministic policies. We find that of the $4! = 24$ possible policy orderings, 12 are realizable via some reward function. In each of those 12 orderings, exactly two policies (of the six available pairs of policies in the ordering) can be set to equal value without resulting in the trivial reward function (*which* pair can be equated depends on the ordering in consideration). Attempting to set three policies equal always results in the trivial reward simplification.

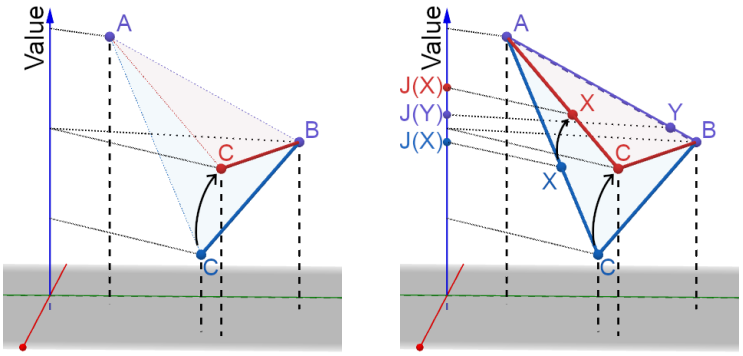
For example, given the ordering $\pi_{00} \leq \pi_{01} \leq \pi_{11} \leq \pi_{10}$, the simplification $\pi_{00} = \pi_{01} < \pi_{11} < \pi_{10}$ is represented by $R = \left[\begin{smallmatrix} 0 & 3 \\ 2 & 1 \end{smallmatrix}\right]$, where $\mathcal{R}(s, a) = R[s, a]$: for example, here taking action 1 from state 0 gives reward $\mathcal{R}(0, 1) = 3$. But there is no reward function representing a non-trivial simplification of this ordering with $\pi_{01} = \pi_{11}$. We develop and release a software suite to compute these results. Given an environment and a set of policies, it can calculate all policy orderings represented by some reward function. Also, for a given policy ordering, it can calculate all nontrivial simplifications and reward functions that represent them. For a link to the repository, as well as a full exploration of these policies, orderings, and simplifications, see Appendix D.3.

5.3 Unhackability in Infinite Policy Sets

The results in Section 5.1 do not characterize unhackability for infinite policy sets that do not contain open sets. Here, we provide two examples of such policy sets; one of them admits unhackable reward pairs and the other does not. Consider policies A, B, C, and reward functions $\mathcal{R}_1$ with $J_1(C) < J_1(B) < J_1(A)$ and $\mathcal{R}_2$ with $J_2(C) = J_2(B) < J_2(A)$. Policy sets $\Pi_a = \{A\} \cup \{\lambda B + (1 - \lambda)C : \lambda \in [0, 1]\}$ and $\Pi_b = \{A\} \cup \{\lambda B + (1 - \lambda)C : \lambda \in [0, 1]\}$ are depicted in Figure 5; the vertical axis represents policies' values according to $\mathcal{R}_1$ and $\mathcal{R}_2$. For Π_a, $\mathcal{R}_2$ is a simplification of $\mathcal{R}_1$, but for Π_b, it is not, since $J_1(X) < J_1(Y)$ and $J_2(X) > J_2(Y)$.

6 Discussion

We reflect on our results and identify limitations in Section 6.1. In Section 6.2, we discuss how our work can inform discussions about the appropriateness, potential risks, and limitations of using of reward functions as specifications of desired behavior.



(a) (b)

Figure 5: Infinite policy sets that do not contain open sets sometimes allow simplification (a), but not always (b). Points A, B, C represent deterministic policies, while the bold lines between them represent stochastic policies. The y-axis gives the values of the policies according to reward functions $\mathcal{R}_1$ and $\mathcal{R}_2$. We attempt to simplify $\mathcal{R}_1$ by rotating the reward function such that $J_2(B) = J_2(C)$; in the figure, we instead (equivalently) rotate the triangle along the AB axis, leading to the red triangle. In (a), $\mathcal{R}_2$ simplifies $\mathcal{R}_1$, setting all policies along the BC segment equal in value (but still lower than A). In (b), $\mathcal{R}_2$ swaps the relative value of policies X and Y ($J_1(X) < J_1(Y) = J_2(Y) < J_2(X)$) and so does not simplify $\mathcal{R}_1$.

6.1 Limitations

Our work has a number of limitations. We have only considered finite MDPs and Markov reward functions, leaving more general environments for future work. While we characterized hackability and simplification for finite policy sets, the conditions for simplification are somewhat opaque, and our characterization of infinite policy sets remains incomplete.

As previously discussed, our definition of hackability is strict, arguably too strict. Nonetheless, we believe that understanding the consequences of this strict definition is an important starting point for further theoretical work in this area.

The main issue with the strictness of our definition has to do with the symmetric nature of hackability. The existence of complex behaviors that yield low proxy reward and high true reward is much less concerning than the reverse, as these behaviors are unlikely to be discovered while optimizing the proxy. For example, it is very unlikely that our agent would solve climate change in the course of learning how to wash dishes. Note that the existence of *simple* behaviors with low proxy reward and high true reward *is* concerning; these could arise early in training, leading us to trust the proxy, only to later see the true reward decrease as the proxy is further optimized. To account for this issue, future work should explore more realistic assumptions about the probability of encountering a given sequence of policies when optimizing the proxy, and measure hackability in proportion to this probability.

We could allow for approximate unhackability by only considering pairs of policies ranked differently by the true and proxy reward functions as evidence of hacking iff their value according to the true reward function differs by more than some ε. Probabilistic unhackability could be defined by looking at the number of misordered policies; this would seem to require making assumptions about the probability of encountering a given policy when optimizing the proxy.

Finally, while unhackability is a guarantee that no hacking will occur, *hackability* is far from a guarantee of hacking. Extensive empirical work is necessary to better understand the factors that influence the occurrence and severity of reward hacking in practice.

6.2 Implications

How should we specify our preferences for AI systems' behavior? And how detailed a specification is required to achieve a good outcome? In reinforcement learning, the goal of maximizing (some) reward function is often taken for granted, but a number of authors have expressed reservations about